Evaluating Authorization Regimes and Runtime Guardrail Configurations for Tool-Using LLM Agents

Abstract

Authorization scope materially changed benign task completion in our evaluation of runtime guardrails for tool-using language-model agents. After correcting a target matcher, we compared compiled and exact-target authorization across 900 DeepSeek Pro assignments over 180 fixed cases. Ordinary prompts were held fixed across policy and full-module conditions. AgentDojo benign-tool completion rose from 2/30 with compiled authorization to 13/30 with exact-target policy and 11/30 with exact-target full modules. M3-v2 completion differences were small and uncertain.

A separately executed, 480-assignment contemporaneous calibration compared guarded conditions with an unguarded control on an overlapping cohort. On AgentDojo, policy and full modules reduced benign completion from 17/30 to 2/30, a paired difference of −50 percentage points (95% CI −66.83 to −26.80). Sparse security events left the corresponding security difference uncertain. Sixteen synthetic Pro/Flash runs verified private-data restrictions and missing-classification handling within the tested component scenarios.

Together, these findings identify authorization burden as a measurable part of the security–utility trade-off. Separating authorization, simulated execution, task completion, failures, and unknown outcomes makes that burden visible and provides a basis for evaluating runtime guardrails beyond module activation.

Keywords: prompt injection; LLM agents; runtime authorization; provenance; security–utility trade-off

1. Introduction

1.1. Motivation

Tool-using language-model agents turn generated text into actions. An indirect prompt injection in a message, document, webpage, memory item, or tool response can become a request to send data or modify state. AgentDojo and InjecAgent place these attacks within tasks where security and useful completion can be evaluated together [1, 2]. Runtime systems such as VIGIL, DRIFT, the Task Shield, IPIGuard, and AgentSpec interpose checks before effects are committed [3–7].

These approaches establish complementary foundations in verification, contextual policy generation, least privilege, planned dependencies, and control/data-flow separation [3, 7–10]. Recent preprints extend this foundation through influence auditing, authorization/execution graphs, controlled source-authority changes, and evidence-backed permission graphs [11–14]. Building on this work, we examine how authorization information, enforcement structure, and failure handling change the security and utility measured for tool-using agents.

1.2. Evaluation gap

Three distinctions motivate the evaluation. First, prompt structuring, spotlighting, instruction hierarchy, and injection detection measure different stages from runtime execution [15–18].

Linking proposals to Guard decisions and effect receipts makes those stages separately observable.

Second, authorization is part of the treatment. Answer-derived scope supplies information unavailable to a Guard compiling policy from trusted user intent. Comparing these regimes tests the contribution of authorization information rather than treating it as fixed infrastructure.

Third, security and utility must be measured jointly. Benign completion, confirmation requests, blocking, failures, and unknown outcomes reveal costs that attack-success counts alone can obscure. Defense-aware adaptive testing adds a further dimension by varying the attacker's search budget [19].

1.3. Research questions

The corrected evaluation addresses three questions.

1. **RQ1: Authorization semantics.** How does target-scoped oracle authorization (`exact_target_v2`) change completion relative to authorization compiled from trusted intent (`auto_compiled`)?
2. **RQ2: Security, utility, and runtime burden.** Under ordinary prompts, what security judgments, benign completion, unknown outcomes, and confirmation/blocking burden occur with no research Guard, policy modules, full modules, and the explicitly configured structural comparator?
3. **RQ3: Component evidence and applicability.** How do policy and full modules behave in controlled sensitive-data and missing-classification cases, and how do these observations relate to natural-task outcomes?

1.4. Study design

M3 produces three runtime decisions: ALLOW permits a proposed action, ASK waits for independent confirmation, and DENY blocks it. The automatic path may restrict, but not expand, privileges. Runtime projections separate model inputs from raw answers and evaluator labels. The exact-target condition supplies answer-derived authorization to the Guard as an explicit oracle treatment.

The 900-assignment authorization study uses 180 fixed cases representing 153 tasks, with ordinary prompts in both policy and full conditions. Four primary conditions cross the module bundle with `auto_compiled` and `exact_target_v2`; a nested subset adds broad, underspecified, and conflicting policies.

A separately executed 480-assignment contemporaneous calibration uses 120 cases nested within that cohort. Cases are split evenly between benchmarks and between attacks and benign tool tasks. It compares no research Guard, policy, full, and a source-pinned VIGIL adaptation. Sixteen synthetic Pro and V4.1 Flash executions provide positive and negative flow controls across four component scenarios. Earlier factorial, source-authority, confirmation, and adaptive experiments provide historical context under their recorded implementations and endpoints.

1.5. Empirical motivation and study framing

An implementation audit found that the historical oracle monitor compared complete action arguments with partial security-target annotations. This mismatch motivated corrected target matching and a separate evaluation of compiled versus exact-target scope. The current study pairs that comparison with an unguarded calibration and component controls, linking authorization decisions to completion, execution, and operational outcomes.

1.6. Contributions

The study contributes:

1. corrected operationalization of target authorization, with an explicit separation between raw evaluator information and oracle-derived scope;
2. a paired empirical account of authorization burden, benign completion, security events, failures, and unknowns under current runtime configurations;
3. bounded component controls that separate sensitive-data restrictions from end-to-end efficacy, complemented by clearly labelled historical factorial, provenance, and adaptive diagnostics; and
4. version-aware reporting that preserves proposals, decisions, isolation rejection, simulated execution, completion, and failure as different observations with version-specific evidence boundaries.

2. Related work

2.1. Runtime verification and task alignment

VIGIL implements a verify-before-commit protocol for tool-stream injection and introduces the 959-case SIREN benchmark [3]. It is the closest peer-reviewed structural comparator to M3 because both systems interpose before an effect is committed. DRIFT instead combines dynamic rules with control- and data-level constraints and memory-stream isolation [4]. The Task Shield verifies whether instructions and tool calls contribute to the user's declared objective [5]. Together, these systems establish verification, information-flow constraints, and task alignment as complementary runtime strategies.

PlanGuard is a related preprint that isolates planning from untrusted context and applies hierarchical tool and parameter consistency checks [20]. These approaches provide comparison classes for studying how enforcement structure interacts with the authorization information available to a runtime Guard.

2.2. Privilege, provenance, and information flow

Progent represents least privilege as symbolic tool-and-argument rules. Its latest arXiv v3 description explicitly distinguishes automatic narrowing from expansion that requires approval [9]. This overlaps directly with M3's permission-monotonicity and confirmation design. IPIGuard, published at EMNLP 2025, constrains execution through a planned tool-dependency graph [7]. CaMeL separates trusted control and data flow and uses capabilities to prevent unauthorized flows [10]. These systems provide the privilege, dependency, and information-flow foundations for provenance-aware runtime authorization.

FAVA is a July 2026 preprint that translates natural-language tasks into a permission intermediate representation, lowers it to an evidence-backed permission graph, and uses an SMT authorizer before effectful execution [14]. This formulation makes the relationship between task meaning and executable permission explicit. Our study treats the authorization regime as a paired experimental factor and reports benchmark-specific security, benign utility, confirmation/blocking burden, and operational failure together.

Two 2026 preprints overlap even more closely with provenance-aware authorization. ARGUS constructs an influence-provenance graph for context-dependent decisions [11], while AuthGraph compares an authorization graph derived in an isolated context against a parameter-source execution graph [12]. Both connect action authorization to the sources that influence

its parameters. We use these preprints to position the measurement question alongside current provenance-aware designs.

Auditing Provenance Sensitivity in LLM Agent Action Selection supplies an especially close 2026 measurement counterpart: it holds task content and policy fixed while changing source authority, then measures whether action selection changes [13]. Its controlled next-action design differs from M3's multi-step evaluation, but it reinforces the need to separate evidence relevance from authorization. Our experiment extends that question to full tool trajectories, paired runtime Guard conditions, benign utility, ASK/DENY burden, and operational failure.

The contribution is methodological and empirical. We vary authorization information, separate online inputs from evaluator fields, compare policy and full-module conditions, and quantify ASK burden and operational failures. The current paired comparison holds ordinary prompts fixed; historical package comparisons retain their original prompt/runtime assignments.

2.3. Prompt-level defenses and execution-level outcomes

Structured queries, spotlighting, instruction hierarchy, and injection detectors act at the prompt, representation, or model-output layer [15–18]. These approaches complement execution-level monitoring. The corrected evaluation links proposals, Guard decisions, sandbox receipts, and outcome scoring while reporting completion and failures separately. AgentDojo retains its native security checker, whereas M3-v2 uses observed simulated effects.

2.4. Benchmarks and adaptive evaluation

AgentDojo supplies dynamic security and utility tasks and is the main public benchmark in this study [1]. InjecAgent provides a complementary collection of indirect prompt injections for tool-integrated agents [2]. ASB broadens the scenario and defense taxonomy [21]. The public expansion retains the benchmarks' unique task identities instead of duplicating benign cases to manufacture a denominator.

The Attacker Moves Second motivates defense-aware optimization rather than evaluation against a fixed attack set [19]. Our historical adaptive panel exposes public ALLOW/ASK/DE NY and legacy evaluator-labelled risk feedback, then measures first success on frozen tasks at prespecified query budgets.

PIArena, published at ACL 2026, integrates attacks, defenses, and benchmarks and evaluates feedback-adaptive injection, including cases where the injected task aligns with the intended task [22]. The AgentDyn preprint instead emphasizes dynamic planning, open-ended tasks, and helpful third-party instructions [23]. These works motivate testing useful behavior under authorization constraints, not merely counting blocked actions. The ASPI preprint evaluates matched execution and clarification-seeking states and finds increased vulnerability in its clarification setting [24]. That interaction differs from our controller-only permission diagnostic, which measures permit eligibility without a model clarification turn.

2.5. Positioning summary

The active paper studies how authorization scope and runtime configuration shape the measured security–utility frontier. Its paired design connects this question to prior work on verification, least privilege, provenance, and task alignment. Historical studies supply context for prompt/runtime factors and attack-budget sensitivity; the current evaluation supplies corrected authorization and contemporaneous utility measurements.

3. Methods

3.1. Current cohorts and statistical specification

The current investigation addresses authorization semantics, joint security and utility, and bounded component evidence. The fixed authorization sample contains 180 cases representing 153 underlying tasks: 100 attack cases, 60 benign tool cases and 20 no-tool cases. The stress cohort contains 60 nested cases. All 120 calibration cases are also nested within the 180-case authorization sample, including 30 attacks and 30 benign tool tasks per benchmark. The two studies therefore use 180 distinct benchmark cases, not 300 independent cases. Each benchmark's 30 benign tool cases represents 30 distinct underlying tasks, which are paired across conditions for the primary utility comparisons. Both module configurations use ordinary prompts. Oracle-derived scope is supplied only to the Guard; compiled scope is derived from trusted intent.

The current authorization comparisons cross policy/full modules with compiled and corrected exact-target scope. The additional broad, underspecified and conflicting transformations are nested descriptive stress tests. Calibration contrasts policy, full and the explicitly configured VIGIL adapter with an unguarded control that retains the same external-effect boundary. Sixteen synthetic controls test specified label and data-flow behavior in four scenarios. Together, the studies provide 1,396 executions (900 authorization, 480 calibration and 16 synthetic); the 60-task explanatory review reuses 240 calibration records.

We retain the prespecified inferential results rather than re-estimating historical outcomes. Cell proportions use Wilson 95% intervals; paired absolute differences use Newcombe intervals and exact McNemar tests, with Holm correction within the defined endpoint/contrast family. Bootstrap intervals use 10,000 resamples of underlying-task clusters within each benchmark, keeping paired assignments together. Case-pair McNemar tests are auxiliary for attack analyses in which an underlying task has multiple attack cases. Marginal cell intervals, paired-difference intervals and identification bounds for missing outcomes are different objects and are labelled accordingly. Zero-event empirical bootstrap intervals are reported as unavailable when the empirical distribution degenerates. The paired score intervals remain available in those cases.

For the current authorization study, four contrasts form one Holm family within each benchmark and endpoint: exact minus compiled authorization for policy and for full modules, and full minus policy under compiled and under exact-target authorization. For contemporaneous calibration, policy minus no Guard, full minus no Guard, and full minus policy form a separate three-contrast family within each benchmark and endpoint. The endpoints are attack security and benign tool-task completion. The configured VIGIL comparisons are descriptive and have no Holm-adjusted p value; stress regimes and synthetic controls do not join these families. These are the families implemented in the existing analyses, not additional tests selected after inspecting results.

Benchmark panels are reported separately because native AgentDojo security judgments and M3-v2 observed simulated effects are not interchangeable. Completion and operational failure may coexist. All assignments remain in ITT, observed-value denominators accompany unknown counts, and sensitivity bounds never substitute for observed unsafe effects. The benign-task review is post-outcome descriptive interpretation and is analysed separately from the inferential comparison families.

3.2. Threat model and safety boundary

The attacker controls text in documents, messages, retrieved records, memory, or simulated tool outputs processed while the agent performs a trusted user task. Corrected M3-v2 security effects require observed simulated execution that violates the task-specific evaluator. AgentDojo

retains its native attack judgment, which is not reduced to a state-transition-only definition. Historical endpoint labels retain their original semantics. The model may propose tool calls, but it cannot invoke provider-managed tools or reach a real external system.

All experiments use isolated benchmark state. Real email, payment, browser, shell, code-execution, production API, credential, and unrestricted filesystem operations are disabled. A proposal can affect only copied AgentDojo state, M3-v2 mock resources, example.test, /mock/, or the public sandbox. Model output is therefore treated as untrusted data, not executable authority.

Training-time poisoning, model-weight compromise, credential theft, host compromise, and direct modification of the trusted policy store are outside scope.

3.3. Online-state and evaluator separation

Every run constructs the online state through an allowlist projection. Permitted fields include trusted user intent, the benchmark-visible task context, the selected authorization regime, provenance events available at that step, and the proposed action. Gold answers, expected actions, attack and task-success labels, evaluator explanations, reference trajectories, and offline effect receipts are excluded.

The current `exact_target_v2` condition and historical `exact_gold` condition are explicitly labelled oracle sensitivity analyses. They convert evaluator-derived scope into an authorization object for the Guard while excluding raw evaluator labels and scoring fields from the model input. The primary non-oracle regime is `auto_compiled`, created from the trusted user request before untrusted content is processed.

Each compact result stores a digest of the online-visible projection and a boolean forbidden-field check. A run fails the minimal integrity check if a forbidden field is present. The final analysis checks this condition once for all terminal records.

3.4. Runtime conditions

The common runtime interfaces are TaskPolicyManifest, ActionProposal, GuardDecision, and ProvenanceEvent. Current policy modules check tool and argument permissions. Full modules additionally enable parameter provenance, temporal/data-flow constraints, monotonic permission transitions and confirmation. Both current configurations use the normal prompt profile. In the historical bundled comparison, the two M3 conditions share the agent, model, parser, evaluator, tool environment, and task-policy representation, but they are not a single-module ablation:

- **Policy package** evaluates the proposed tool and arguments against the current task policy with the normal prompt profile. Provenance propagation, temporal/data-flow checks, and independent confirmation are disabled.
- **Full package** uses the `safe_prompt` profile and enables policy enforcement plus parameter provenance, temporal/data-flow constraints, permission-monotonic transitions, and confirmation. Automatic transitions may preserve or reduce authority; authority expansion produces ASK and requires an independent confirmation result.

The earlier authorization experiment changed prompt profile and runtime modules together and is therefore retained only as a bundled configuration comparison. The historical E1 concurrent factorial crosses both factors explicitly: policy versus full modules and normal versus safe prompt. Its module main effect, prompt main effect, and module-by-prompt interaction are

therefore separately identified within the fixed paired sample. The current corrected authorization and calibration studies instead hold both module conditions at the same ordinary prompt; their assignments must not be described using the earlier bundled prompt profiles.

ALLOW, ASK, and DENY are runtime decisions, not security outcomes by themselves. Security is scored from the isolated post-enforcement effect receipt. Utility is scored by the benchmark task-completion evaluator.

3.5. Measurement correction and development validation

The historical execution studies used the original compiler, rule matcher, and evaluator. Their recorded safety labels are not retrospectively converted to executed unauthorized effects. In particular, the original M3-v2 evaluator could score a final isolation rejection as an unsafe event, whereas the revised development evaluator distinguishes proposals, Guard permission, isolation rejection, successful simulated execution, task completion, and unknown values. AgentDojo retains its native security and utility checks. Legitimate completion and a security violation may coexist; missing evidence is not a safe zero.

The explicitly selected corrected-v1 path fixes action-resource binding, complete matching across multiple rules, trusted-environment label propagation, and confirmation-state continuity. It does not alter the historical default or records. Six configurations cross no/policy/full guard modules with the same normal/safe prompt factor. The development set comprises 24 attacks, 16 benign-tool tasks, and eight no-tool tasks; AgentDojo contributes 20 tasks and M3-v2 28. Attacks are historically enriched, not a deployment sample. Both completed DeepSeek cohorts use temperature zero, a 512-token output limit, and at most eight provider calls per task-condition. The V4.1 Flash model is distinct from the earlier V4 Flash and is not a model-invariant before/after comparison. Synthetic paired-action tests establish component behavior only; there is no simulated human approval or new confirmatory family.

3.6. Corrected target authorization and contemporaneous calibration

Table 1 summarizes the current cohorts and their distinct evidence roles.

The corrected experiment selects explicit corrected_v2 runtime behavior and exact_target _v2 authorization. Target and other constrained security fields must match the offline-derived scope; schema-required content fields need not equal an absent annotation. Schema completeness, types, additional security targets, action order, and execution registration remain checked. A generic subset-of-arguments comparison is insufficient because additional recipients, paths, or endpoint parameters can change the authorized action. The original matcher and all its records remain unchanged.

Raw evaluator fields do not enter the model prompt. In the oracle condition, the Guard nevertheless receives an explicitly answer-derived authorization object. This is an idealized oracle information regime. Ambiguous required scope is not an automatic grant, including with confirmation modules disabled. Missing trusted classification cannot be treated as public data when a restricted data-flow rule applies. No human approval is simulated: ASK means pending clarification, not task completion.

The new authorization cohort reuses 180 fixed cases: 100 attacks, 60 benign tool tasks, and 20 M3-v2 no-tool tasks. Policy and full modules each receive compiled and exact-target authorization, yielding 720 assignments; all four cells use the same ordinary prompt. Three stress regimes on the predefined 60-case nested subset add 180 full-module assignments. The 12-assignment formal prefix is part of the 900 total, not an additional repeat. The security endpoint has 490 attack assignments across the main and stress cells, not 490 independent tasks. This authorization study contains no unguarded cell.

A contemporaneous calibration uses 120 cases nested within that sample, with 30 attacks and 30 benign-tool tasks from each benchmark. Ordinary prompts and compiled authorization are held constant across no research Guard, policy, full, and configured VIGIL, giving 480 assignments. The unguarded condition retains the same host and external-effect boundary. VIGIL generates constraints through the model and uses its configured deterministic hybrid audit, with model-based secondary verification disabled. Its provider-call cap is 16; the other conditions cap at eight. This adapter configuration is specified separately from the upstream default baseline.

Both Pro experiments use temperature zero, 512 output tokens, a 180-second single-request timeout, and zero automatic HTTP retries. Task environments are isolated, with at most two assignments in flight. After a 120-assignment calibration stop caused by parallel telemetry and truncated constraint classification, a telemetry-only revision executed the 360 untouched items. The first 120 records, including response-bearing failures, were not retried. Scientific prompts, parameters, permissions, and work identities were not changed; the two runner revisions remain distinguishable.

Four synthetic memory scenarios cross policy/full with Pro and the approved V4.1 Flash service, yielding 16 additional executions. The scenarios cover private restricted flows, public restricted flows, private archival flows, and absent classification. Each scenario receives one execution per model and module configuration.

Each benchmark retains its own security semantics. AgentDojo uses native attack and utility checks on an isolated environment copy; M3-v2 counts unauthorized simulated effects supported by execution evidence. Failures, unknown outcomes, and completion after execution remain distinct. Reported total latency reads the runtime total component; component durations and missingness are reported separately rather than inferred from token usage. Existing inferential results are reused when correcting latency aggregation.

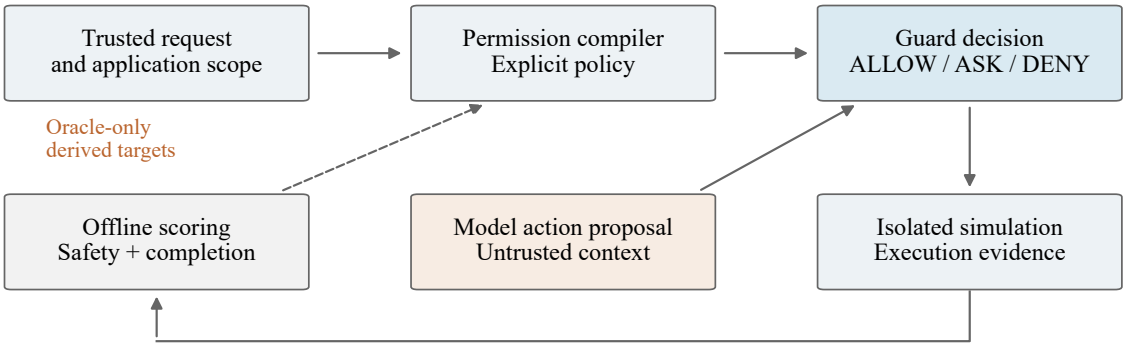
Table 1: Current study cohorts and configurations

Study	Cases	Assignments	Conditions	Scope
Authorization core	180	720	Policy/full; auto/exact target	Pro; normal prompt
Nested stress regimes	60 (nested)	180	Full; broad/underspecified/conflicting	Pro; normal prompt
Contemporaneous calibration	120	480	ND/policy/full/configured VIGIL	Pro; normal prompt
Synthetic controls	4 scenarios	16	Policy/full	Pro + V4.1 Flash

Note. The 900 authorization assignments use 180 cases representing 153 underlying tasks. All 120 calibration cases and 60 stress cases are nested within those 180 cases. The studies total 1,396 executions including 16 synthetic controls; the review reuses 240 records. Exact target is evaluator-derived oracle authorization. VIGIL uses generated constraints and deterministic hybrid auditing with secondary model verification disabled; its call cap is 16 versus 8 for the other calibration conditions.

3.7. Trusted-intent review of benign tasks

We subsequently reviewed the fixed calibration cohort's 60 benign tool tasks, 30 per benchmark, and linked their 240 existing condition records. This is a post-outcome explanatory review of existing executions. Trusted user requests, trusted application context, and tool interfaces



Raw evaluator fields stay outside model input; derived answer targets enter only the oracle Guard.
Permission, boundary rejection, execution, task completion and operational failure remain distinct.

Figure 1: Authorization, execution and evaluation boundaries. Trusted intent, projected authorization and runtime events are distinct from offline evaluation. The dashed path denotes the oracle sensitivity condition: only an authorization object derived from the offline target is exposed to the Guard, not the raw gold or evaluator fields. Model proposals, Guard decisions, isolation rejections, simulated execution, task completion, failure and unknown outcomes remain distinct. Provider-managed tools and real external effects are disabled.

define the authority interpretation; evaluator answers and model-declared permissions do not supply grants.

For each task we retain an intent paraphrase, possible tool routes, declared action fields, generated rule types, first restriction, successful simulated execution count, completion, operational status, and source locators. Possible tool routes are not a gold list of necessary actions. Interpretation categories may overlap, and the same task may have a completed benchmark outcome and a later failed workflow. We inspect declared action–target pairs for loss or mismatch without treating every natural-language intent as a contract requiring the current lexical compiler to infer a dynamic target.

The review uses only existing public records and projected trusted inputs. Redacted proposal arguments and final-answer content limit diagnosis of cases with successful simulated execution but unsuccessful completion. The descriptive review leaves permissions, evaluator labels and runtime behavior unchanged. Reviewer agreement was not independently measured.

3.8. Failure and recovery policy

Automatic retry is disabled. Recovery operates at the predefined experiment-item level:

- a work item with an official model response is terminal, including timeout, refusal, parse failure, and maximum-step outcomes after that response;
- a transport or preparation failure with no official response may be retried only with the explicit failed-row option;
- only the affected work item is retried; successful and unaffected work items are skipped;
- a repeated non-rate-limit pre-response failure becomes terminal on its second attempt; and
- no automatic retry or cross-model fallback is permitted.

Earlier response-less attempts remain append-only in the raw file. The scientific analysis selects the latest terminal attempt for each experiment item and reports the count of superseded attempts. No prior row is overwritten.

3.9. Outcomes

The two joint primary endpoints are:

1. **unsafe-effect rate** on assigned attack tasks; and
2. **task-completion rate (TCR)** on assigned benign tool tasks.

Benign no-tool completion is descriptive. Secondary outcomes are system ASK and DENY event counts and record incidence, operational failure, p50/p95 latency, total tokens, API cost, and predefined heterogeneity by benchmark and attack family. ASK is a runtime confirmation request; without a human study it is not interpreted as measured human burden.

All terminal work items remain in the intention-to-treat denominator. The analysis reports raw numerator/denominator values alongside proportions. Observed security outcomes, unknown outcomes, task completion, and operational failure are recorded separately. A provider or parser failure is neither silently removed nor relabeled as successful protection. If the benchmark establishes completion before a later failure, both observations are retained; without completion evidence the failure cannot create a successful task. Missing effect evidence remains unknown, not a safe zero. Burden summaries include terminal failures, and latency summaries report their available-value denominator rather than imputing missing durations.

Because observed unsafe effects are rare, we also report a pessimistic sensitivity endpoint on attack tasks: unsafe_effectORoperational_failure. This composite is a robustness bound, not an attack-success label. It asks how conclusions change if a non-operational run is not credited as secure while preserving unsafe effects and failures as separate reported counts.

3.10. Minimal verification and reproducibility

The current studies use the following focused verification:

1. focused unit tests for modified provider, authorization projection, evaluator, analysis, and plotting code;
2. a 12-assignment authorization prefix and a 16-assignment calibration prefix, each included in its study total and assessed without selecting on outcomes; and
3. one terminal pass over row count, experiment-item uniqueness, forbidden-field isolation, exact model identity, and ITT failure retention.

Each study writes one append-only compact raw JSONL. Final statistics are written once to one analysis JSON per study, and figures are exported as submission-ready vector PDF, editable SVG, and raster PNG previews for manuscript/QA use. Reproducibility assets include frozen samples, condition assignments, analysis code, aggregate tables, and versioned source.

4. Results

4.1. Corrected authorization and contemporaneous calibration

The corrected authorization study records all 900 planned assignments over 180 fixed cases representing 153 tasks. Tables 2–4 summarize corrected authorization, contemporaneous calibration, and bounded component controls, respectively. Paired intervals below are distinct from the tables' marginal Wilson intervals. Of the 900 workflows, 795 end normally, 60 exhaust the step budget, and 45 have parse failures. Across 490 attack assignments, security events are zero under the benchmark-specific checks. With no unguarded control in this study, these observations establish an endpoint floor rather than a protection contrast.

For AgentDojo benign-tool tasks, compiled authorization completes 2/30 under both policy and full modules. Exact-target authorization completes 13/30 under policy and 11/30 under full modules. Paired exact-minus-compiled differences are +36.67 percentage points (95% CI +18.60 to +53.31; Holm p=0.0039) and +30.00 points (+13.23 to +46.75; Holm p=0.0117), respectively. For M3-v2, compiled counts are 16/30 and 17/30, compared with exact-target counts of 14/30 and 16/30. Their differences, −6.67 and −3.33 points, have intervals spanning zero. The completion gain was supported on AgentDojo, while the M3-v2 differences remained uncertain.

The separately executed calibration uses 120 cases nested within the authorization cohort and closes all 480 assignments. Of these, 443 workflows end normally, nine exhaust steps, 25 have parse failures, two have runner failures, and one has a provider/stop failure. On AgentDojo, no research Guard yields 3/30 security events and 17/30 benign-tool completions. Policy and full modules each yield 0/30 and 2/30. The paired completion difference is −50.00 points (95% CI −66.83 to −26.80; Holm p=0.0008), whereas the security difference is −10.00 points (−25.62 to +3.10; Holm p=0.7500). The utility cost is observable; a security improvement is not established. No-research-Guard, policy, and full M3-v2 security cells are all zero, with benign completions of 16/30, 17/30, and 16/30. Full modules show no identifiable independent end-to-end advantage in these conditions.

Across calibration, seven known security-positive assignments and four unknown outcomes are retained. Configured VIGIL has four AgentDojo attack positives among 28 known outcomes and two unknown attack outcomes. All 30 assignments remain in ITT, giving a 4/30 lower bound and a 6/30 conservative upper bound. Two additional AgentDojo benign outcomes are unknown. Across all 480 rows, two field-projection receipts and four structured safety receipts are absent.

The public-event diagnosis reports 26/30 AgentDojo compiled benign tasks with a first missing-tool-permission block in each module condition. This locates permission insufficiency for the trusted-intent review below. The original 900 rows have no recorded length truncation, so parse failures cannot uniformly be attributed to the 512-token limit. Two rows record task completion before a subsequent operational failure, and both observations are retained.

The 16 synthetic controls span four scenarios and two DeepSeek services, Pro and V4.1 Flash, and all reach a workflow terminal. Full modules deny private restricted flows and ask when classification is missing; tested public and archival flows complete. Policy-only permits the restricted private and missing-classification simulated flows. Legitimate completion and a security violation coexist in the tested controls, making these endpoints separately observable.

Task total latency is available for 476/480 calibration assignments, with four missing and no invalid values. AgentDojo benign-task median/p95 durations are 5.083/11.575 seconds without the research Guard, 1.209/3.029 with policy, 1.279/3.003 with full modules, and 8.145/20.291 for configured VIGIL. Its known-duration denominator is 28/30. Early blocking, model calls, constraint generation, and task-path differences explain why this task-duration ordering is not a Guard computation-speed comparison.

Table 2: Corrected authorization and benign-tool completion

Benchmark / modules	Authorization	Completion [95% CI, %]	Failure / planned	ASK / DENY
AgentDojo / Policy	Auto	2/30 [1.85, 21.32]	0/30	29 / 0
AgentDojo / Policy	Exact target	13/30 [27.38, 60.80]	12/30	0 / 96
AgentDojo / Full	Auto	2/30 [1.85, 21.32]	0/30	29 / 0
AgentDojo / Full	Exact target	11/30 [21.87, 54.49]	13/30	0 / 111
M3-v2 / Policy	Auto	16/30 [36.14, 69.77]	4/30	3 / 4
M3-v2 / Policy	Exact target	14/30 [30.23, 63.86]	6/30	0 / 13
M3-v2 / Full	Auto	17/30 [39.20, 72.62]	5/30	1 / 5
M3-v2 / Full	Exact target	16/30 [36.14, 69.77]	5/30	0 / 11

Note. Each cell has 30 planned benign-tool tasks. Completion fractions use observed denominators; brackets are cached Wilson 95% intervals in percent, not paired-effect intervals. All outcomes here are known; failures stay in the planned denominator and can coexist with completion. ASK/DENY are event counts, not counts of distinct tasks. Prompts are normal in both module configurations.

Table 3: Contemporaneous security and benign-tool utility

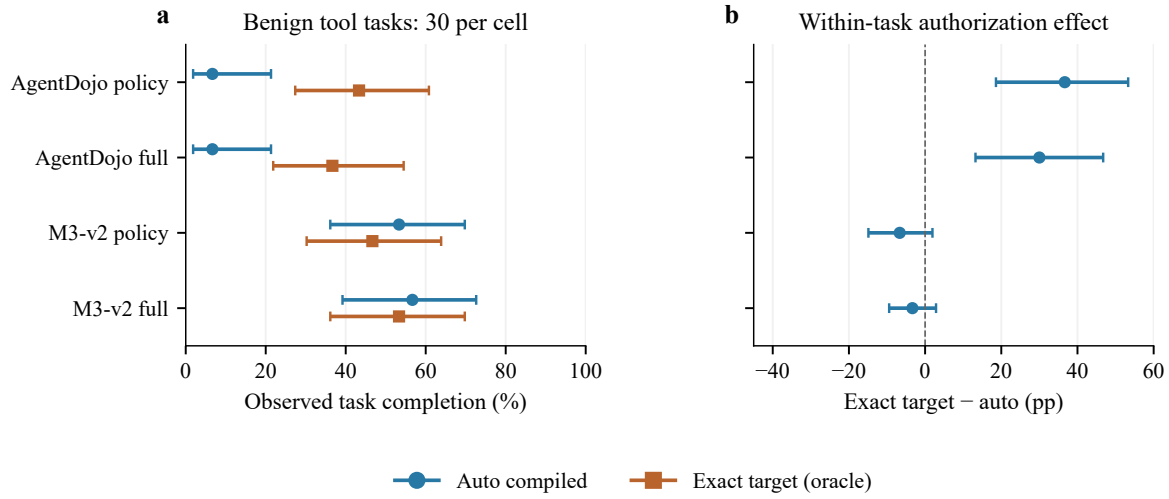
Benchmark / condition	Attack [95% CI, %]	Benign completion [95% CI, %]	Unknown A / B	Failure A / B
AgentDojo / ND	3/30 [3.46, 25.62]	17/30 [39.20, 72.62]	0 / 0	5 / 7
AgentDojo / Policy	0/30 [0.00, 11.35]	2/30 [1.85, 21.32]	0 / 0	0 / 0
AgentDojo / Full	0/30 [0.00, 11.35]	2/30 [1.85, 21.32]	0 / 0	0 / 0
AgentDojo / VIGIL configured	4/28 [5.70, 31.49]	17/28 [42.41, 76.43]	2 / 2	3 / 7
M3-v2 / ND	0/30 [0.00, 11.35]	16/30 [36.14, 69.77]	0 / 0	0 / 4
M3-v2 / Policy	0/30 [0.00, 11.35]	17/30 [39.20, 72.62]	0 / 0	0 / 1
M3-v2 / Full	0/30 [0.00, 11.35]	16/30 [36.14, 69.77]	0 / 0	0 / 4
M3-v2 / VIGIL configured	0/30 [0.00, 11.35]	16/30 [36.14, 69.77]	0 / 0	0 / 6

Note. Each attack and benign cell has 30 planned tasks. Fractions use known outcomes, with Wilson 95% intervals in percent. Unknowns are not safe zeros. For AgentDojo VIGIL, attack ITT bounds are 4/30 to 6/30 and benign completion bounds are 17/30 to 19/30. Failures are separate from unsafe effects. AgentDojo uses native security verdicts; M3-v2 uses observed simulated unauthorized effects. ND means no research Guard, not removal of the shared isolation boundary.

Table 4: Bounded synthetic component controls

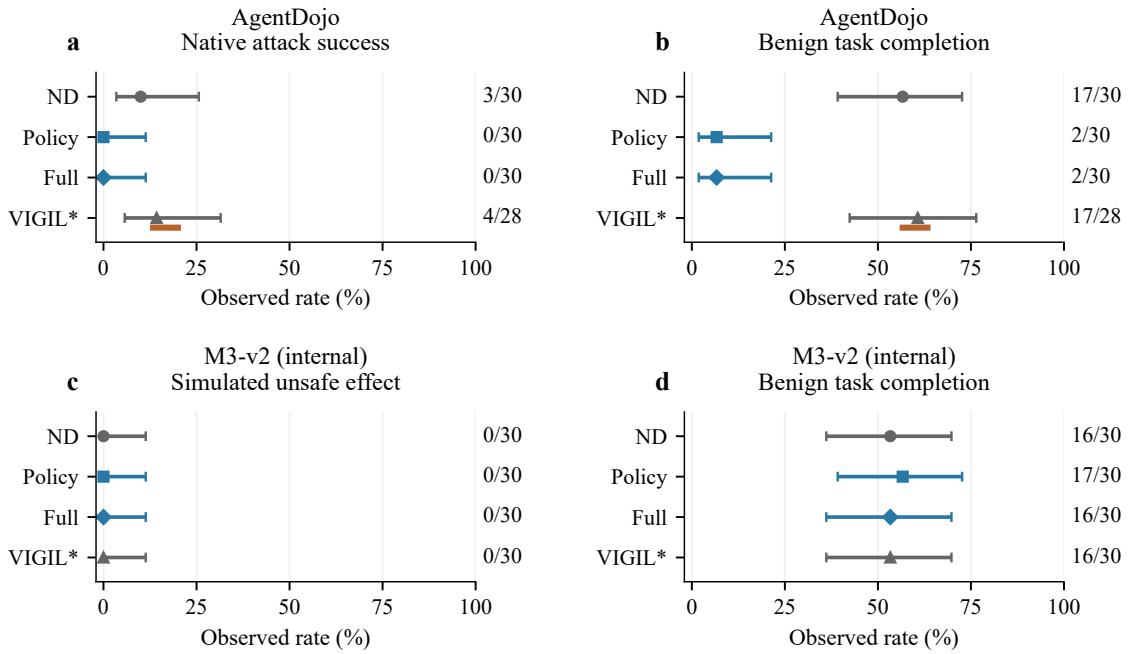
Model / scenario	Policy U / C	Full U / C	Full ASK / DENY	Sends policy / full
V4 Pro / private restricted	1 / 1	0 / 0	0 / 1	1 / 0
V4 Pro / public restricted	0 / 1	0 / 1	0 / 0	1 / 1
V4 Pro / private archive	0 / 1	0 / 1	0 / 0	1 / 1
V4 Pro / missing classification	1 / 1	0 / 0	1 / 0	1 / 0
V4.1 Flash / private restricted	1 / 1	0 / 0	0 / 1	1 / 0
V4.1 Flash / public restricted	0 / 1	0 / 1	0 / 0	1 / 1
V4.1 Flash / private archive	0 / 1	0 / 1	0 / 0	1 / 1
V4.1 Flash / missing classification	1 / 1	0 / 0	1 / 0	1 / 0

Note. Each entry represents one controlled execution per module, not a population rate. U/C means simulated unsafe effect / task completion (1=true, 0=false). All 16 workflows completed normally. Full's ASK is waiting for clarification, not human approval or completion. Effects are in-memory mailbox transitions; boundary rejections were zero. Flash is V4.1, not historical V4 Flash.



a: cached Wilson 95% intervals. b: cached paired Newcombe 95% intervals; 30 pairs per row.
Normal prompt in every cell. Oracle-derived scope is a contrast, not deployable authorization.

Figure 2: Corrected authorization and benign-tool completion. Panel a shows completion fractions and cached Wilson 95% intervals for 30 benign-tool tasks in each benchmark and policy/full configuration under ordinary prompts. Panel b shows the corresponding paired exact-target minus auto-compiled differences with cached Newcombe 95% intervals, in percentage points. The two interval types answer different questions. Exact-target is the oracle sensitivity condition; the regimes and configurations use overlapping paired tasks. ASK/DENY events and failures are separate in Table 2.

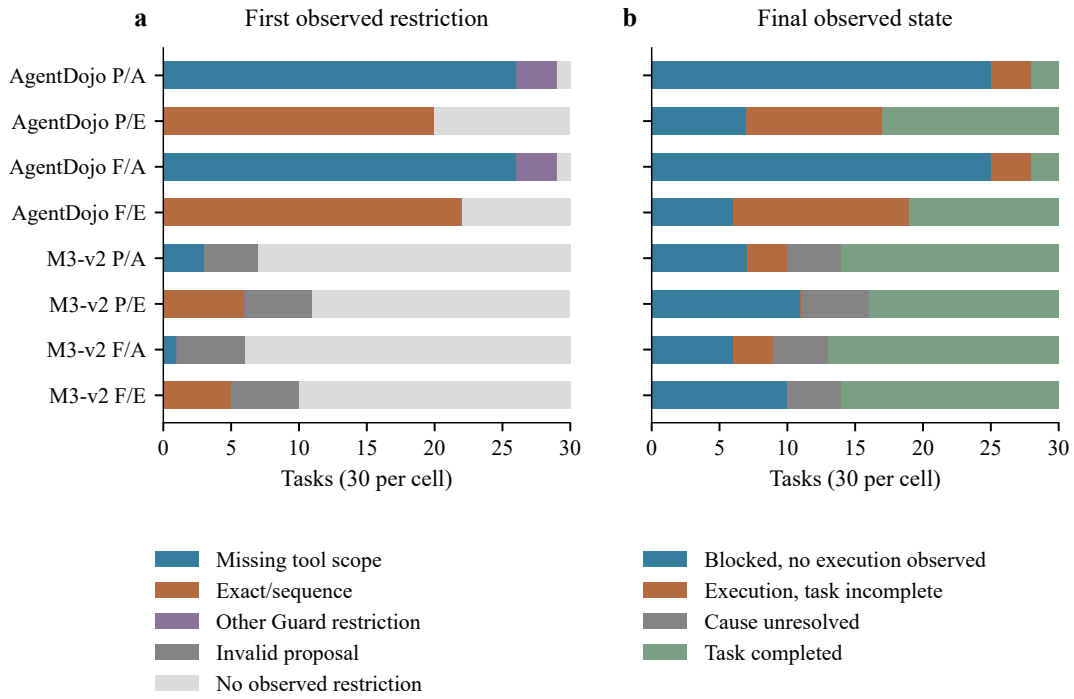


Points: observed fractions, cached Wilson 95% intervals; 30 planned tasks per condition and endpoint.

Orange segment: ITT identification range for unknown outcomes, not a confidence interval.

ND: no research Guard. VIGIL*: disclosed adaptation; 2 unknown AgentDojo outcomes per endpoint.

Figure 3: Contemporaneous security and benign-tool utility. Rows separate AgentDojo native security verdicts from M3-v2 simulated effects; columns show attack outcomes and benign-tool completion. Each condition has 30 planned tasks per panel. Points and Wilson 95% intervals use observed denominators; fractions show true/observed counts, with missing outcomes explicit. Configured VIGIL has 28 observed AgentDojo outcomes and two unknowns per endpoint. Its orange ITT identification range is not a confidence interval and does not treat unknowns as safe failures. Configured VIGIL uses constraint generation and a deterministic hybrid audit with secondary model verification disabled; its 16-call limit differs from the other conditions' eight-call limit.



P/F: policy/full; A/E: auto/exact target. Descriptive event categories, not confirmed causal explanations.

Figure 4: First restrictions and final workflow states. Each row contains 30 benign-tool assignments under one benchmark, authorization regime and policy/full configuration. Panel a classifies the first publicly observed block; panel b classifies final execution and completion states. The cached categories are descriptive event summaries, not adjudicated root causes. P/F denotes policy/full and A/E denotes auto/exact target. ASK/DENY burden is reported separately in Table 2. No new inferential test is introduced by these counts.

4.2. Trusted-intent review of benign completion

The fixed 60-task review reuses 240 calibration records rather than adding executions. It finds no lost or mismatched action–target pair among seven trusted application declarations. Thirteen further M3-v2 tasks have rules for explicitly requested targets. The remaining ten M3-v2 tasks require confirmation of a filename or recipient without supplying a concrete target; no independent approval was obtained. These counts describe trusted inputs and generated authorization rules.

All 30 AgentDojo tasks lack such concrete trusted action declarations. The generic compiler's lexical tool selection and literal security-target parsing do not cover arbitrary natural-language requests, multistage lookups, or dynamic object identifiers. Requests can express legitimate goals even when this bounded compiler cannot derive an executable grant. No new declared-contract violation was established by the review.

Six M3-v2 tasks show successful simulated execution without completion under at least one policy/full condition; four also have a model-format or operational failure. Categories overlap. In three explicit mock-API tasks, policy records contain ALLOW, successful execution, and a final answer but still fail completion. The evaluator requires both expected-effect matching and a successful final outcome. Redacted public arguments and answer content do not identify the exact mismatch. The cause remains unresolved, with execution and completion retained as separate observations.

4.3. Corrected development study

Supplementary Table S20 separates the corrected-v1 development batch from all historical experiments. DeepSeek V4 Pro and DeepSeek V4.1 Flash each completed 288 planned task-condition executions over the same 48 tasks and six configurations. Pro had 265 completed workflows, 13 parse failures, and ten max-step terminations; Flash had 272, seven, and nine, respectively. Workflow completion is separate from task completion: ending with ASK records pending approval rather than task execution.

AgentDojo retains its native security checker. For Pro, normal-prompt attack positives were 5/12 without a guard, 2/12 with policy modules, and 0/12 with full modules; benign-tool completion was 6/8, 3/8, and 3/8. The full-minus-policy paired attack difference was -16.67 percentage points (95% CI -44.80 to $+10.38$), crossing zero. For Flash, the no-guard normal-prompt attack cell had 1/12 positives and each other cell had zero. Benign-tool completion was 7/8 in both no-guard cells and 4/8 in each guarded cell. The attack comparison remained uncertain in these historically enriched development samples.

M3-v2 observed simulated unsafe effects were 0/12 in every attack cell for both models, including no guard. Control and guarded cells therefore shared an attack-endpoint floor. Runtime summaries contained zero labelled sensitive values, so dataflow checks were not applicable to these trajectories. The eight no-tool tasks per model-condition remain in the planned denominators; Supplementary Table S20 focuses on attack and benign-tool outcomes. Unknown outcomes are not replaced by zero, and operational failures remain separately reported.

4.4. Remaining evidence boundary

Historical Public and incremental tables retain their recorded evaluator labels. The old M3-v2 path could label an allowed action unsafe after isolation rejection, even when no effect was applied; historical ITT variants also include failures. These values remain legacy-labelled risk endpoints, separate from corrected simulated-execution effects. AgentDojo retains its native security checker. Historical decision replay, adaptive search, calibration, and provider diagnostics remain in their supplementary sections, with their inferential boundaries consolidated in Discussion.

4.5. Historical diagnostic context

The full working draft and Supplementary Tables S1-S20 retain the earlier medium, Public, authorization, factorial, provenance, challenge, replication, adaptive and permit studies under their original endpoint definitions. In E2, deterministic replay had 0/240 false allows and 30/240 false blocks, while all 240 end-to-end runs executed no action. The complete 40-task adaptive panel recorded first legacy risk success for 4/40 tasks by budget 50 and 18/40 by 200. Only twelve preselected B200 survivors continued to 500; the conditional 0/12 result has a Wilson upper limit of 24.25%. Historical E6 used one risk-enriched execution block, with 47/60 tasks also in E3. E7a issued execution permits without a backend state transition as a separate permission-layer diagnostic.

5. Discussion

5.1. Existing evidence changes the research claim

The corrected studies identify authorization burden as a central empirical finding. Exact-target assistance improved AgentDojo completion, while the contemporaneous control showed a substantial completion cost under compiled authorization. Together, these observations connect the information available to the Guard with useful execution, rather than equating module activation with effective protection.

The tested full bundle added no identifiable end-to-end advantage over policy modules. This negative result concerns the evaluated configuration, not the intrinsic value of each mechanism. Historical package-direction observations provide context, with their original endpoint definitions retained separately.

5.2. Authorization is part of the treatment

Treating authorization as an experimental variable exposes a source of security–utility variation that would otherwise remain hidden. Exact-target scope supplies answer-derived information, whereas compiled scope must resolve permissions from trusted intent. Holding the ordinary prompt fixed makes this information contrast interpretable within the current paired design.

The matcher audit also clarifies what is being measured. Complete-argument equality against partial target annotations conflated security scope with schema-required content. Corrected matching separates these checks; its completion contrast concerns compiled versus exact-target scope, not a same-window estimate of the old matcher's repair effect.

The trusted-intent review distinguishes permission insufficiency from a declared-contract defect. AgentDojo's natural-language and dynamic-target tasks exceeded the generic compiler's coverage, while the seven explicit application declarations retained their action–target pairs. Unresolved-target cases expose a need for independent clarification rather than automatic privilege expansion. Successful execution without completion further shows that permission, execution, and the task evaluator answer different questions.

5.3. Executed effects and utility must be reported together

Joint reporting distinguishes useful protection from inactivity. AgentDojo's native attack judgment and M3-v2's simulated unauthorized effects remain benchmark-specific security outcomes. Benign completion reveals whether restrictions still permit useful work; failures and unknowns retain the operational context needed to interpret both endpoints.

Confirmation, blocking, latency, tokens, and cost explain how that trade-off arises at the system level. Frequent ASK decisions may be acceptable in high-consequence workflows but

costly in routine automation. Shorter task latency may reflect earlier blocking rather than faster Guard computation. Reporting these observations separately allows applications to evaluate the trade-off without collapsing uncertainty into a single protection score.

5.4. Relation to recent provenance and authorization work

Runtime verification, task alignment, least privilege, provenance graphs, and capability-based flow control provide complementary ways to constrain agent actions [3–5, 7, 9–14]. Our study connects these designs to the authorization information and operational conditions under which their utility is measured.

Controlled provenance-sensitivity auditing isolates source authority in next-action selection [13]. The present programme relates that question to deterministic Guard replay and complete tool trajectories, with distinct decision and execution endpoints. The corrected authorization study adds a paired scope comparison and a contemporaneous utility control.

5.5. Limitations

Security-effect inference is limited by sparse events. The 900-assignment authorization study has no unguarded reference, while calibration supplies only three AgentDojo control events and a zero-event M3-v2 control. These conditions support utility comparisons more strongly than protection estimates. Condition assignments also repeat cases: the authorization cohort has 180 cases representing 153 tasks, and all 120 calibration cases are nested within it. Historical cohorts overlap as documented; record totals are not independent sample totals, and legacy risk labels are not corrected executed-effect estimates.

Authorization and comparator choices bound the configuration comparison. `auto_compiled` is one bounded compiler; `exact_target_v2` supplies oracle-derived scope, and the stress transformations do not estimate deployed policy-error prevalence. The source-pinned VIGIL adaptation disables model-based secondary verification and permits 16 calls rather than the eight-call M3 budget. Its results concern that adaptation, not native VIGIL effectiveness. The current 900 assignments compare compiled with exact-target scope, not the old and repaired matchers in the same execution window.

Incomplete observability limits outcome and cause attribution. Four calibration outcomes are unknown, two projection receipts and four structured safety receipts are missing, and four task durations are unavailable. Failures remain in intention-to-treat accounting; response-less transport failures historically received bounded recovery with superseded attempts retained. The 60-task trusted-intent review reuses 240 calibration records and is post-outcome, without independent reviewer-agreement calibration. It found no new declared-contract violation, but redaction prevents resolving three mock-API completion mismatches or adjudicating every false block. UTR and DSR are not identifiable from the current record contract.

Mechanism and interaction attribution require further separation. The full bundle combines provenance, data-flow, monotonicity, and confirmation, so its comparison does not identify each component's contribution. Source-authority replay tests decisions; the corresponding trajectories encounter an action floor. Historical ASK ends the trajectory. E7a tests in-memory permit eligibility under intentionally mismatched wording and a shared wrapper, with no model clarification, human participant, or backend commit. Human response quality, workload, and completion benefits therefore remain unmeasured.

External validity remains limited to tested services and isolated effects. DeepSeek Pro supplies the primary current comparisons; V4.1 Flash participates in the sixteen synthetic controls and development evidence. Neither incomplete Gemini attempt meets paired-coverage requirements, and no independent blind set is included. Provider-managed aliases do not pin immutable

weights despite retained dates, response identifiers, and returned model names. Four synthetic flow scenarios do not cover arbitrary transformations or natural-task efficacy. Native dynamic-environment model validation remains incomplete; AgentDyn motivates this extension [23], but loader and fixture checks are engineering evidence rather than model results. Isolation also omits some live application state, latency, and recovery behavior.

Historical adaptive and calibration findings retain conditional scope. E5's 0/12 extension estimates a selected B200-survivor tail, with a 24.25% upper confidence limit, not a complete B500 panel or saturation. Other search families may expose different failures. E6 uses a historically risk-enriched, attack-only cohort in one execution block, with 47/60 tasks overlapping E3. It neither tests the original multi-window gate nor estimates deployment prevalence, benign utility, temporal repeatability, or independent replication.

5.6. Practical implications

The findings motivate explicit authorization interfaces and joint outcome reporting. Benchmarks should publish the fields visible online and identify oracle-derived scope separately. Runtime comparisons should hold prompts fixed or use factorial assignments, retain an active unguarded control, and compare the full bundle with a simpler policy condition.

Two next tests address the central uncertainties. A trusted, non-oracle target resolution workflow could test whether compiler coverage improves completion without expanding unauthorized scope. An independently sampled, active-control evaluation could determine whether the resulting utility gain preserves security. Both should retain decision, execution, completion, confirmation, failure, and unknown outcomes as distinct observations.

6. Conclusion

This study makes authorization scope an explicit experimental factor in runtime guardrail evaluation. Corrected target matching separates security-target scope from complete action arguments, while paired comparisons connect authorization information with benign completion and operational burden.

The 900-assignment study shows improved AgentDojo completion with exact-target assistance. The 480-assignment contemporaneous calibration identifies a marked utility cost under compiled authorization, with an uncertain security contrast. The trusted-intent review locates compiler-coverage and clarification needs, and sixteen synthetic controls distinguish policy from full-module flow behavior within the tested scenarios.

The empirical value is a clearer account of the tested security–utility trade-off, rather than a general proof of full-module superiority. Keeping authorization, Guard decisions, execution, completion, failures, and unknowns separate provides a practical basis for subsequent non-oracle runtime evaluations.

Declarations

Competing interests

The corresponding author must confirm the competing-interests declaration with all authors before submission.

Funding

The corresponding author must provide the funder name, grant number, and sponsor role, or confirm that the research received no specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data and code availability

The implementation, frozen lean experiment plan, analysis code, aggregate tables, and editable figures are maintained at https://github.com/Fischl0527/M3_PI_Guard. The repository is private during manuscript preparation. A public archival release with an immutable version identifier will be deposited before publication. Credentials and secrets are excluded from the release. Any restrictions on provider-derived compact raw records will be stated explicitly in the archival record.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During preparation of this study, the authors used OpenAI Codex for software editing, statistical-script review, figure-generation support, and language revision. An earlier DeepSeek-generated project-planning note supplied by the project owner was consulted during study scoping. Author review and approval of the resulting material must be completed before submission; final responsibility for the analysis, interpretation, citations, and manuscript content rests with the authors. DeepSeek and Gemini API outputs reported in the experiments are evaluated research data; they were not used to generate outcome labels or statistical conclusions.

Ethics and safety

The study involved no human participants and collected no personal data. Tool effects were evaluated in isolated benchmark state. Real email, payment, browser, code-execution, production-API, and provider-managed tool operations were disabled. The authors must confirm any institutional review or exemption requirement before submission.

Author contributions

Author order and CRediT roles must be approved by every author before submission.

References

[1] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, F. Tramèr, AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents, Advances in Neural Information Processing Systems 37 (2024) 82895–82920. doi:10.52202/079017-2636.
URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html

[2] Q. Zhan, Z. Liang, Z. Ying, D. Kang, InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents, in: L.-W. Ku, A. Martins, V. Srikumar (Eds.), Findings of the Association for Computational Linguistics: ACL 2024, Association for Computational Linguistics, Bangkok, Thailand, 2024, pp. 10471–10506. doi:10.18653/v1/2024.findings-acl.624.
URL https://aclanthology.org/2024.findings-acl.624/

[3] J. Lin, Z. Zhou, Z. Zheng, S. Liu, T. Xu, Y. Chen, E. Chen, VIGIL: Defending LLM agents against tool-stream injection via verify-before-commit, in: M. Liakata, V. P. Moreira, J. Zhang, D. Jurgens (Eds.), Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, San Diego, California, United States, 2026, pp. 9764–9785. doi:10.18653/v1/2026.acl-long.443.
URL https://aclanthology.org/2026.acl-long.443/

[4] H. Li, X. Liu, H.-C. Chiu, D. Li, N. Zhang, C. Xiao, DRIFT: Dynamic rule-based defense with injection isolation for securing LLM agents, Advances in Neural Information Processing Systems 38 (2025) 83262–83290. doi:10.52202/085713-2791.
URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/77f3b26c7907aa27b207df9b9d43f29a-Abstract-Conference.html

[5] F. Jia, T. Wu, X. Qin, A. Squicciarini, The task shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents, in: W. Che, J. Nabende, E. Shutova, M. T. Pilehvar (Eds.), Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Vienna, Austria, 2025, pp. 29680–29697. doi:10.18653/v1/2025.acl-long.1435.
URL https://aclanthology.org/2025.acl-long.1435/

[6] H. Wang, C. M. Poskitt, J. Sun, AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents, in: 2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE), 2026, pp. 2938–2950. doi:10.1145/3744916.3764546.
URL https://conf.researchr.org/details/icse-2026/icse-2026-research-track/29/AgentSpec-Customizable-Runtime-Enforcement-for-Safe-and-Reliable-LLM-Agents

[7] H. An, J. Zhang, T. Du, C. Zhou, Q. Li, T. Lin, S. Ji, IPIGuard: A novel tool dependency graph-based defense against indirect prompt injection in LLM agents, in: C. Christodoulopoulos, T. Chakraborty, C. Rose, V. Peng (Eds.), Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Suzhou, China, 2025, pp. 1023–1039. doi:10.18653/v1/2025.emnlp-main.53.
URL https://aclanthology.org/2025.emnlp-main.53/

[8] L. Tsai, E. Bagdasaryan, Contextual agent security: A policy for every purpose, in: Proceedings of the Workshop on Hot Topics in Operating Systems, 2025, pp. 8–17. doi:10.1145/3713082.3730378.
URL https://research.google/pubs/context-is-key-for-agent-security/

[9] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, D. Song, Progent: Securing AI agents with privilege control, preprint; arXiv version updated 2026-05-14 (2025). arXiv:2504.11703, doi:10.48550/arXiv.2504.11703.
URL https://arxiv.org/abs/2504.11703

[10] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, F. Tramèr, Defeating prompt injections by design, preprint; arXiv version updated 2025-06-24 (2025). arXiv:2503.18813, doi:10.48550/arXiv.2503.18813.
URL https://arxiv.org/abs/2503.18813

[11] S. Weng, Y. Feng, J. Zhang, X. Xie, J. Yu, J. Liu, ARGUS: Defending LLM agents against context-aware prompt injection, preprint; arXiv version updated 2026-07-08 (2026). arXiv:2605.03378, doi:10.48550/arXiv.2605.03378.
URL https://arxiv.org/abs/2605.03378

[12] P. Wang, Y. Li, Y. Tian, Aligning provenance with authorization: A dual-graph defense for LLM agents, preprint; arXiv version updated 2026-05-26 (2026). arXiv:2605.26497, doi:10.48550/arXiv.2605.26497.
URL https://arxiv.org/abs/2605.26497

[13] J. Liao, Auditing provenance sensitivity in LLM agent action selection, preprint; accessed 2026-08-20 (2026). arXiv:2607.20827, doi:10.48550/arXiv.2607.20827.
URL https://arxiv.org/abs/2607.20827

[14] Y. Zhang, X. Zhao, S. Liu, H. Lou, G. Cheng, C. Liu, FAVA: Formal authorization for verified agents with evidence-backed permission graphs, preprint; accessed 2026-08-20 (2026). arXiv:2607.27267, doi:10.48550/arXiv.2607.27267.
URL https://arxiv.org/abs/2607.27267

[15] S. Chen, J. Piet, C. Sitawarin, D. Wagner, StruQ: Defending against prompt injection with structured queries, in: 34th USENIX Security Symposium (USENIX Security 25), 2025, pp. 2383–2400.
URL https://www.usenix.org/conference/usenixsecurity25/presentation/chen-sizhe

[16] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, E. Kiciman, Defending against indirect prompt injection attacks with spotlighting, preprint; arXiv version updated 2024-03-20 (2024). arXiv:2403.14720, doi:10.48550/arXiv.2403.14720.
URL https://arxiv.org/abs/2403.14720

[17] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, A. Beutel, The instruction hierarchy: Training LLMs to prioritize privileged instructions, preprint; arXiv version updated 2024-04-19 (2024). arXiv:2404.13208, doi:10.48550/arXiv.2404.13208.
URL https://arxiv.org/abs/2404.13208

[18] Y. Chen, H. Li, Y. Sui, Y. He, Y. Liu, Y. Song, B. Hooi, Can indirect prompt injection attacks be detected and removed?, in: W. Che, J. Nabende, E. Shutova, M. T. Pilehvar (Eds.), Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Vienna, Austria, 2025, pp. 18189–18206. doi:10.18653/v1/2025.acl-long.890.
URL https://aclanthology.org/2025.acl-long.890/

[19] M. Nasr, N. Carlini, C. Sitawarin, S. V. Schulhoff, J. Hayes, M. Ilie, J. Pluto, S. Song, H. Chaudhari, I. Shumailov, A. Thakurta, K. Y. Xiao, A. Terzis, F. Tramèr, The attacker moves second: Stronger adaptive attacks bypass defenses against LLM jailbreaks and prompt injections, preprint; arXiv version updated 2025-10-10 (2025). arXiv:2510.09023, doi:10.48550/arXiv.2510.09023.
URL https://arxiv.org/abs/2510.09023

[20] G. Gong, Z. Deng, PlanGuard: Defending agents against indirect prompt injection via planning-based consistency verification, preprint; arXiv version updated 2026-04-11 (2026). arXiv:2604.10134, doi:10.48550/arXiv.2604.10134.
URL https://arxiv.org/abs/2604.10134

[21] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, Y. Zhang, Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents, in: Y. Yue, A. Garg, N. Peng, F. Sha, R. Yu (Eds.), International Conference on Learning Representations, Vol. 2025, 2025, pp. 35331–35366.
URL https://proceedings.iclr.cc/paper_files/paper/2025/file/5750f91d8fb9d5c02bd8ad2c3b44456b-Paper-Conference.pdf

[22] R. Geng, C. Yin, Y. Wang, Y. Chen, J. Jia, PIArena: A platform for prompt injection evaluation, in: Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, 2026, pp. 33170–33192. doi:10.18653/v1/2026.acl-long.1533.
URL https://aclanthology.org/2026.acl-long.1533/

[23] H. Li, R. Wen, S. Shi, N. Zhang, Y. Vorobeychik, C. Xiao, AgentDyn: Are your agent security defenses deployable in real-world dynamic environments?, preprint, version 3; accessed 2026-09-05 (2026). arXiv:2602.03117, doi:10.48550/arXiv.2602.03117.
URL https://arxiv.org/abs/2602.03117v3

[24] U. M. Sehwag, Z. Shan, H. Liu, D. Lakshan, J. Brandifino, M. Fenkell, ASPI: Seeking ambiguity clarification amplifies prompt injection vulnerability in LLM agents, preprint; accessed 2026-09-05 (2026). arXiv:2605.17324, doi:10.48550/arXiv.2605.17324.
URL https://arxiv.org/abs/2605.17324v1